StreamLab

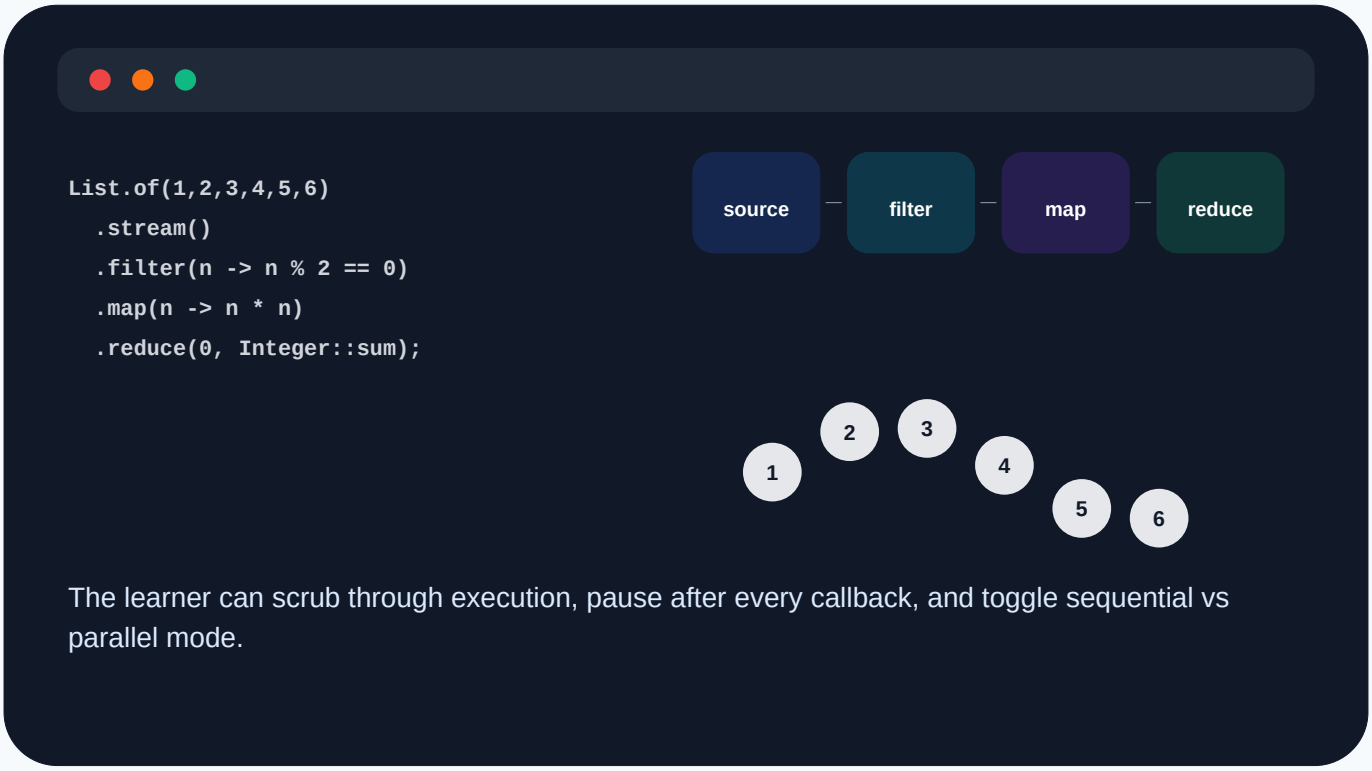
Java Stream API Visual Lab

A Next.js and TypeScript app that simulates how Java streams execute behind the curtain.

The idea

StreamLab lets a learner build a Java Stream pipeline and watch each element move through map, filter, sorted, distinct, limit, reduce, collect, and other operations. Instead of treating streams as invisible magic, it opens the engine room: lazy evaluation, intermediate vs terminal operations, spliterator behavior, short-circuiting, and parallel execution are animated step by step.

100% TypeScript | Next.js | JVM simulation | Parallel streams



The learner can scrub through execution, pause after every callback, and toggle sequential vs parallel mode.

What the app teaches

Lazy execution

Intermediate operations do not run when declared. They are stored as a pipeline plan until a terminal operation asks for results.

Element-by-element flow

Most pipelines process one element through several stages before pulling the next element. The app shows this as moving tokens.

Intermediate vs terminal

filter, map, peek, sorted and distinct transform the plan. forEach, collect, count and reduce trigger actual traversal.

Short-circuiting

findFirst, anyMatch and limit can stop traversal early. The visualizer highlights the exact moment execution ends.

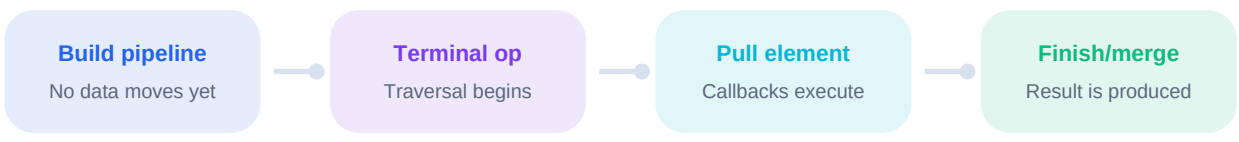
Stateful operations

sorted and distinct may need buffering. The app shows the hidden basket where elements wait before continuing.

Parallel behavior

Parallel mode splits data into tasks, runs worker lanes, then merges results according to operation semantics.

Core mental model



Main product experience

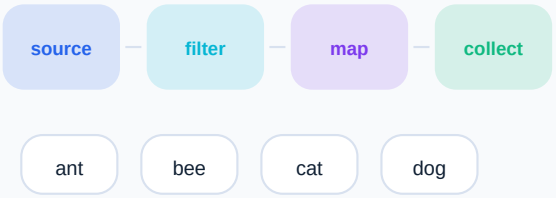
StreamLab workspace

Code, pipeline graph, timeline, thread lanes, and JVM notes in one teaching cockpit.

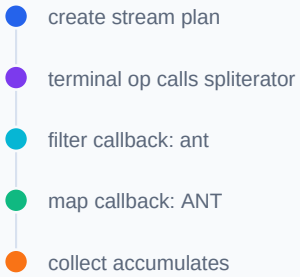
1. Pipeline editor

```
List.of("ant", "bee", "cat",
"dog")
.stream()
.filter(s -> s.length() == 3)
.map(String::toUpperCase)
.collect(toList());
```

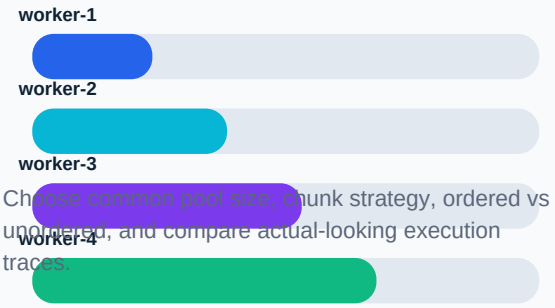
2. Visual pipeline



3. Timeline debugger



4. Parallel mode



Choose common pool size, chunk strategy, ordered vs unordered, and compare actual-looking execution traces.

5. JVM simulation notes

Every animation step can open an explainer card: spliterator.tryAdvance(), sink chain, lambda invocation, buffering for sorted/distinct, fork/join task split, combiner call, encounter order, and why side effects become dangerous in parallel streams.

Execution simulation model

The app does not execute Java bytecode. It implements a faithful teaching simulator in TypeScript: Java-like collections, stream operations, pipeline stages, terminal traversal, and a simplified ForkJoin-style scheduler.

1. **Next.js UI**
   Editor, controls, animation canvas, lesson cards
2. **Stream Simulator Core**
   Pipeline AST, stages, sinks, terminal operations
3. **JVM Model**
   Spliterator, task splitting, buffers, encounter order
4. **Scheduler**
   Sequential runner or simulated worker pool ticks
5. **Trace Store**
   Immutable events used to replay and debug execution

Trace event example

```
{
  tick: 42, thread: "worker-2", element: "cat",
  operation: "filter", callback: "s.length() == 3",
  input: "cat", output: true, nextStage: "map"
}
```

Feature set and learning modes

The lab should feel less like documentation and more like a glass-bottom boat over the Stream API: every callback, buffer, split, and merge becomes visible.

1 Step-through execution

See: One callback at a time, with input and output badges.

Learn: Learners see cause and effect instead of guessing.

2 Sequential vs parallel toggle

See: One lane becomes many worker lanes using the same pipeline.
Learn: Shows why parallel streams change execution shape.

3 Operation library

See: map, filter, flatMap, peek, sorted, distinct, limit, skip, reduce, collect.
Learn: Covers the APIs used in interviews and production code.

4 Stateful buffers

See: Drawers appear for sorted, distinct, grouping, and limit behavior.
Learn: Explains where streaming flow pauses or reshapes.

5 Short-circuit visualizer

See: anyMatch, findFirst, and limit turn on stop lights and cancel remaining work.
Learn: Makes early termination easy to understand.

6 Side-effect warnings

See: Mutation detector and race-condition badges in parallel mode.
Learn: Teaches why shared mutable state is dangerous.

Interview challenge mode

Add small puzzles: predict the output, find the hidden side effect, explain why sorted waits, repair a broken reducer, and compare sequential vs parallel traces. This makes StreamLab useful beyond the demo, turning it into a practice dojo for Java interviews.

MVP scope

Phase 1

Editor presets, sequential animation, 8 core operations

Phase 2

Parallel mode, simulated worker pool, trace replay

Phase 3

Puzzles, shareable demos, portfolio landing page

Why it works as a portfolio project

StreamLab is small enough to build, technical enough to impress, and visual enough for a recruiter or hiring manager to understand in thirty seconds. It shows Java expertise, frontend product skill, systems thinking, and teaching ability in one compact package.

1 Backend credibility

It demonstrates deep knowledge of Java streams, laziness, terminal operations, spliterators, reducers, and parallel execution semantics.

2 Frontend craftsmanship

It uses Next.js, TypeScript, animation state, interaction design, reusable components, and clean data visualization.

3 Education value

It can become a public learning tool for developers preparing interviews or trying to understand modern Java.

4 Demo-friendly

A hiring manager can click one preset, toggle parallel mode, and instantly see a non-trivial technical concept come alive.

Suggested positioning

StreamLab: Java Pipeline Visualization
A TypeScript and Next.js learning lab that simulates Java Stream API execution, including lazy evaluation, pipeline stages, terminal operations, stateful buffering, and parallel stream behavior through an interactive JVM-inspired visual debugger.

Repository name: streamlab-java-pipeline-visualizer